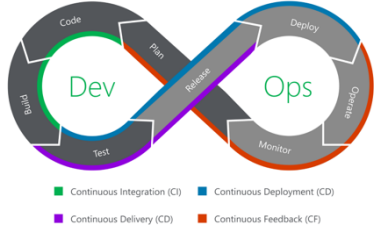


1 Cloud Computing

- **Characteristics:** On-demand self-service, broad network access, resource pooling (multi-tenancy), rapid elasticity, measured service (pay-as-you-go)
- **Service models:** SaaS, PaaS, IaaS
- **Models:** private, community, public, hybrid
- **Failure points web service:** application bugs, deployment failures, scalability issues, infrastructure failures, monitoring gaps, security vulnerabilities
- **Container use challenges:** cultural changes, lack of training, security, CI/CD, monitoring, complexity

2 DevOps

Cultural mindset, Automation-first approach, Continuous processes, Monitoring & feedback loops, Rapid & reliable releases, Security



- **Plan:** requirements, stories creation, architecture, metrics & alerts analysis, risk & rollback strategy
- **Code:** local builds & tests, reviews, gated merges
- **Build:** compile source, create container images, publish artifacts to a registry, generate versioned release packages, store build metadata for traceability
- **Test:** automated unit, integration, E2E, performance & load testing, security/vulnerability scanning, code coverage, quality gates, publish test reports
- **Release:** promote verified artifact to release repository, assign immutable version, publish release notes and notify stakeholders, store release metadata (build ID, commit SHA, artifact hash) for traceability
- **Deploy:** Pull released artifact from repository into target environment, apply deployment manifest, execute deployment strategy, provide / trigger automated rollback if validation fails
- **Operate:** Document, troubleshoot, scale, apply security patches, rotate secrets, manage configuration drift, backup & restore, compliance
- **Monitor:** Define service-level indicators (SLIs) & service-level objectives (SLOs), dashboards, alerting rules, health checks, logging, metrics and traces

2.1 Continuous Lifecycle

- **Continuous development:** Plan & Code
- **Continuous integration:** Code, Build & Test
- **Continuous Testing:** Test
- **Continuous Delivery:** Test & Release
- **Continuous Deployment:** Test, Release & Deploy
- **Strategies:** **Rolling deployment:** update gradually to minimize downtime, **Blue green:** two environments for fast switching, **Canary release:** specify user group to deploy to first, **Feature flags:** deploy but activate later, **Dark Launching:** use real user traffic but hide it
- **Continuous Feedback:** Operate, Monitor & Plan

3 Platform Engineering

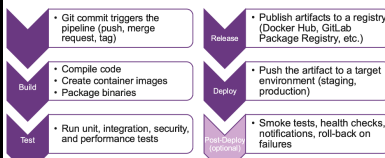
- Cultural mindset to build internal platforms that allow efficient delivery of applications

- **Core responsibilities:** enable self-service, standardize runtimes & tooling, abstract infrastructure, enforce compliance & security, provide observability
- **Typical building blocks from bottom to top:** compute & orchestration, IaC, Pipelines, Deployment, Package & Artifact Management, Policy & Security, Observability, Cost & Capacity, Self-service portal

4 CI/CD

- A pipeline is a series of automated steps that helps teams deliver code faster, safer and more reliably
- Faster feedback loops - higher quality
- Reduce manual effort - fewer human errors
- Consistent environment - no "works on my machine"
- Parallel jobs - scalability to process many commits
- Logged pipeline output - traceability for audits
- Increased efficiency - cost savings
- Enables continuous delivery and deployment

Concept	Purpose
Stage	Logical grouping of jobs stages: [build, test, deploy]
Job	Atomic unit of work that runs inside a stage <job name>: { stage: build, script: [...] }
Step	Individual commands/scripts of a job script: ["cmd1", "cmd2"]
Rule	Controls when a job runs (branch filters, tags, manual triggers, schedules) rules: [{ if: ..., when: ... }]
Service	Auxiliary containers/services (database, Docker-in-Docker) that run alongside a job services: [{name: "postgres15", alias: "db"}]
Runner	Compute resources that execute jobs tags: ["my-mac-runner"]
Cache	Temporary storage that speeds up repeated work (dependency downloads, compiles) cache: { key: "...", paths: ["\$HOME/.cache"] }
Artifact	Files produced by a job that are persisted for later stages or for download by users artifacts: {paths: ["bin"], expire_in: "1 week"}
Registry	Stores for build images or packages script: ["docker login ...", "docker build ...", "docker push ..."]
Variable	Key-value pairs (plain or masked) injected into jobs for configuration or credentials variables: {FOO: "bar"}



- image: go1.18:0 container image for job
- needs: ["job1", "job2"]: complex ordering of jobs, ignores stage ordering
- dependencies: ["job1", "job2"]: for artifact sharing
- **Runners:** Local shell, SSH, Parallels, VirtualBox, Docker, Kubernetes, Instance or Project
- **Typical environments:** local, development, integration, testing/QC, staging/pre-prod, prod

stages: - build - test - deploy build: stage: build script: - echo "x" > artifacts/o.txt artifacts: paths: - artifacts/ expire_in: 1 hour	deploy_staging: stage: deploy environment: name: staging url: https://staging.ch on_stop: stop_staging script: - cat k8s.yaml envsubst kubectl apply -f - artifacts: expire_in: 1 hour stop_staging: stage: deploy environment: name: staging action: stop script: - echo "Stopping staging"
--	---

5 GitOps

- **Push-based (e.g. Jenkins):** triggered by git push, complex drift detection & rollback, easy to use, different targets possible, needs prod admin access, fast, firewall to target must be open, adjustment for new envs needed
- **Pull-based (e.g. ArgoCD):** triggered by agent git pull, automatic change apply, drift detection & rollback, needs prod admin & git access, agent in every cluster needed

- **4 Core Principles:** **Declarative**, **Versioned & Immutable**: desired state in Git
- **Pulled automatically:** agent pulls from git
- **Continuous Reconciliation:** agent continuously observes actual state and attempts to apply desired
- **Side effects:** Documentation & Reproducibility

5.1 ArgoCD

- Platform Engineering approach, Web UI, CLI
- Best for enterprises, multi-cluster, GUI needed
- **Visual dashboard:** app health, sync status/history
- Pre- and post-sync hooks for complex logic, granular access control via SSO, built-in health assessment
- **App of Apps pattern:** keep Applications resources in the Git repo so that they are synced
- **Key Resources:**
 - **Application:** what (which repo), where (cluster & namespace), how (sync policy)
 - **ApplicationSet:** generator that creates multiple Applications based on a template & list of inputs
 - **AppProject:** logical grouping restricts what can be deployed where and who has access to the app

5.2 Flux CD

CLI first approach, no GUI, Helm Apps, Gitless GitOps possible (OCI registry or S3 buckets), GitOps Toolkit, Kubernetes native, low resource footprint

Key Resources:

- **GitRepository / OCIRepository / Bucket / HelmRepository:** resource definition sources
- **Kustomization:** apply manifests / kustomizations
- **HelmRelease:** apply helm charts

5.3 Secrets management

Encrypted secrets: Bitnami Sealed Secrets, Mozilla SOPS, Vault/OpenBao, **References to secrets:** External Secrets Operator, Secrets Store CSI Driver

6 Infrastructure Provisioning

- **Infrastructure as Code (IaC):** managing & provisioning infrastructure through code instead of manually
- **Tools:** HEAT, AWS CloudFormation, Ansible, OpenTofu, Terraform, Pulumi, Crossplane
- **Imperative:** How to achieve result, explicit control flow, stateless, not idempotent
- **Declarative:** What the result should look like, implicit control flow, stateful, idempotent, e.g. OpenTofu

6.1 Terraform / OpenTofu

- Open source IaC engine written in Go
- Uses HashiCorp Configuration Language (HCL)
 - Uses { } for blocks and key="value" for settings
 - Supports simple loops, conditionals and functions
- Linux Foundation's OpenTofu replaces Terraform
- **Workflow:** Write *.tf file, tofu {init, plan, apply, destroy}
- **Providers:** mandatory, translate code into specific cloud API calls, define the building blocks (VMs, DBs, etc.) to use, multi-platform, check if your real-world infrastructure matches code, handle authentication
- **Resources:** specific infrastructure objects (VMs, DBs, etc.), declare what to create/manage, represent the desired state of a component

- **Data:** fetch information from existing infrastructure, external reference, uses data not managed by code, e.g. get AWS AMI based on image name
- **Variables:** dynamic values provided at runtime, allow customizations without changing code, keep secrets and environment-specific data separate, can be set in apply command or Env var, apply command prompts if no default value is set
- **Outputs:** data displayed after successful deployment, passes data like an IP to the user or other modules, highlights critical resource attributes in the CLI
- **References:** connects different blocks of code together, dependency mapping, tells which resource to build first, implicit or explicit with depends_on
- **State:** JSON file mapping code to real-world IDs (e.g. AWS ARN), local file terraform.tfstate, allows to remember what it built previously, identifies if someone manually changed a setting in the console, automatically created first run of apply, 3-way diffs: NEW (code), EXISTING (cloud), PREVIOUS (tfstate)
- **Backends:** state file should be shared to work in teams, e.g. AWS S3, procedure when someone runs apply: acquire lock from S3, fetch tfstate from S3, update cloud, send updated tfstate to S3
- **Modules:** templates for reusability, containers for grouping multiple resources together, enforces standardization and best practices
- **Recomm. files:** terraform.tf, backend.tf, providers.tf, main.tf, variables.tf, outputs.tf, locals.tf, override.tf

variable "instance_type" { default = "t2.micro" }	resource "aws_instance" "web" { ami = "ami-0c55b199cfafef1f0" instance_type = var.instance_type output "public_ip" { value = aws_instance.web.public_ip }
---	--

7 Configuration Management

- After infrastructure provisioning
- Automate system identification, patches & updates
- Centrally modify base configurations and roll them out to all applicable systems
- Identify outdated, poorly performing and noncompliant configurations
- Tools: Ansible, Puppet, Chef, CFEngine, Juju

7.1 Ansible

- **Use cases:** configuration management, security & compliance, provisioning, application deployment, continuous delivery, orchestration
- No state/shared state/locking/lifecycle
- Declarative & imperative hybrid, idempotency opt.
- Templating, agentless, open source
- Connection via localhost, SSH or WINRM
- **Ad-hoc command example:**
- **Not idempotent:** `ansible host1 -m command -a "useradd test"`
- **Idempotent:** `ansible host1 -m user -a "name=test"`

Terms:

- **Inventory:** INI or YAML, list of hosts & variables
- **Library:** many readily available modules
- **Module:** unit of code, task plugin, written in Python
- **Collection:** next-gen modules with version independent of Ansible version, have FQCNC
- **Role:** automatically load related vars, files, tasks, handlers, and other Ansible artifacts based on a known file structure, can easily be shared
- **Tag:** attribute you can set on an Ansible structure (plays, roles, tasks), execute tasks subset instead of whole playbook
- **Task:** unit of work (e.g. copy a file), uses a module

- **Play:** one or more tasks that run in order of definition, defines hosts to run on
- **Playbook:** YAML file, contains one or more plays
- **Facts:** Automatic collection of info about hosts when running a playbook, available in `ansible_facts` Variable
- **Variables:** names should be letters, numbers and _
- **Loops:** declared below task, some plugins can take lists as options, which is better than looping
- **Conditionals:** when clause below tasks, raw Jinja2 expressions, can use `ansible_facts`
- **Handler:** task that is called by another task, calling task must have triggered a change, executed after all other tasks were successful
- **ignore_errors:** don't stop play exec in case of error
- **force_handlers:** run handlers even if tasks are failing
- **Vault:** keeps sensitive data like passwords or keys in encrypted files that can be placed in Git, password can be provided via file, prompt or secret manager
- **Templating:** Jinja2 for dynamic expressions and variables, happens on the Ansible controller, only the information needed is passed to the target machine
- **Jinja2 Filters:** transform data, e.g. formatting, list filters, random number, math filters, JSON query

name: Play1 (whole file = playbook) hosts: web become: true vars: packages: ["nginx", "curl"] enable_service: true secret_pass: "{{ vault_pass }}" roles: - myrole tasks: - name: Install packages apt: name: "{{ item }}" state: present	loop: "{{ packages }}" notify: restart nginx - name: Configure app template: src: app.conf.j2 dest: /etc/app.conf when: enable_service tags: config handlers: - name: restart nginx service: name: nginx state: restarted
--	---

8 Kubernetes

- Open-source system for automating deployment, scaling & management of containerized apps
- **Objects:** persistent entities, records of intent, represent state of cluster
- **Controllers:** tracks at least one resource type / object, makes the current become the desired state
- **Manifest:** declarative YAML to create a resource
`kubectl {create|replace|apply} -f manifest.yaml`
`kubectl get pod x -o yaml --dry-run=client > x.yaml`
- **Pod:** smallest deployable unit, represents processes running, contains one or multiple containers
- **Sidecar:** container in pod that runs concurrently with the main container, provides supporting functionality
- **Init container:** container in pod that completes a task before the regular container is started
- **ReplicaSet:** makes sure that we have a configured number of pod replicas running
- **Deployment:** provides declarative updates to ReplicaSets and Pods (e.g. scaling & update strats)
- **HorizontalPodAutoscaler:** scales number of pod replicas between minReplicas/maxReplicas based on resource-based/custom/external metrics
- **Namespace:** logical isolation within cluster, divide resources among users/teams/projects, RBAC policies/ResourceQuotas/LimitRanges per ns, cluster-wide resources like nodes still shared
- **Probe types:** command exec, httpGet, tcpSocket
- **readinessProbe:** traffic not routed to pod if it fails
- **livenessProbe:** restart pod when it fails
- **Scheduling:** nodeName, nodeSelector, (Anti)Affinity, Pod topology spread constraints, Taints/Tolerations

apiVersion: apps/v1 kind: Deployment metadata: name: app spec:	- name: init image: busybox command: ['sh', '-c', 'sleep 10'] containers:
--	--

